JAVA INTERVIEW PREPARATION

CHAPTER 1

OBJECT-ORIENTED PROGRAMMING
BEYOND THE FOUR DEFINITIONS

Senior / Lead Java Developer Textbook Edition

Full approved reading material - not summarized

Object-Oriented Programming Beyond the Four Definitions

Senior / Lead Java Developer Reading Chapter

Object-oriented programming is commonly introduced through four terms: encapsulation, abstraction, inheritance, and polymorphism. These definitions are useful when learning the language, but they are not enough for a senior Java developer.

At a senior level, object-oriented design is primarily about controlling change. A well-designed object protects its own rules, exposes a clear contract, and limits the number of places that must change when business requirements evolve.

An object should therefore be more than a collection of fields with getters and setters. It should represent a meaningful concept and ensure that the concept always remains valid.

Consider a bank account:

```
public class BankAccount {

    private BigDecimal balance;

    public BankAccount(BigDecimal openingBalance) {
        this.balance = openingBalance;
    }

    public BigDecimal getBalance() {
        return balance;
    }

    public void setBalance(BigDecimal balance) {
        this.balance = balance;
    }
}
```

The field is private, so this class technically uses encapsulation. However, it does not protect the account's business rules.

Any caller can assign an invalid balance:

```
account.setBalance(new BigDecimal("-1000000"));
```

The problem is that the class exposes its internal state without controlling how that state changes. Making a field private is only data hiding. Meaningful encapsulation requires the object to protect its invariants.

An invariant is a condition that must remain true whenever an object is visible to the rest of the application.

A stronger design would be:

```
public final class BankAccount {

    private BigDecimal balance;

    public BankAccount(BigDecimal openingBalance) {
        if (openingBalance.signum() < 0) {
            throw new IllegalArgumentException(
                "Opening balance cannot be negative"
            );
        }

        this.balance = openingBalance;
    }

    public BigDecimal balance() {
        return balance;
    }

    public void deposit(BigDecimal amount) {
        requirePositive(amount);
        balance = balance.add(amount);
    }

    public void withdraw(BigDecimal amount) {
        requirePositive(amount);

        if (amount.compareTo(balance) > 0) {
            throw new InsufficientFundsException();
        }

        balance = balance.subtract(amount);
    }

    private static void requirePositive(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive"
            );
        }
    }
}
```

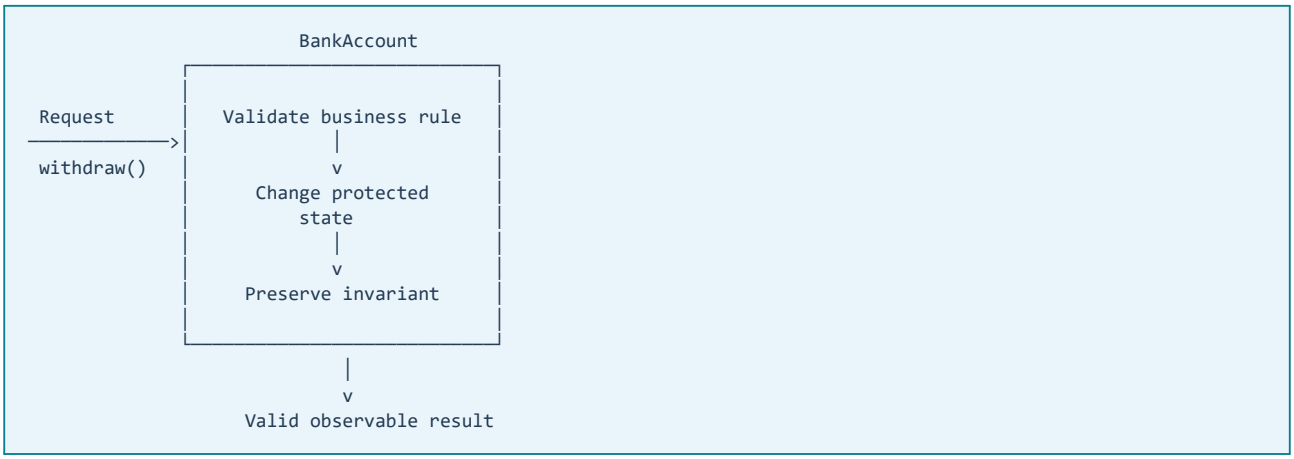
This version does not allow callers to assign an arbitrary balance. Instead, it exposes operations that make sense in the business domain: `deposit` and `withdraw`.

Each operation protects the account's rules. As long as all state changes pass through these operations, the object cannot enter an invalid state accidentally.

The important idea is:

> **Encapsulation means controlling state transitions, not merely hiding fields.**

A Mental Model for Encapsulation



The object acts as a boundary around its state. Callers request an operation, but the object decides whether that operation is valid and how its state should change.

This becomes especially important in large systems. If validation is scattered across controllers, services, and user-interface code, some execution path will eventually bypass it. When an invariant belongs to a domain object, the object itself should normally help protect it.

Abstraction as a Stable Contract

Abstraction means presenting the behavior a caller needs while hiding details the caller should not depend on.

For example:

```
public interface PaymentProcessor {

    PaymentResult process(PaymentRequest request);
}
```

A service using this interface does not need to know whether a payment is processed through a REST API, a message broker, or a third-party SDK.

```
public class CheckoutService {

    private final PaymentProcessor paymentProcessor;

    public CheckoutService(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }

    public Order checkout(Cart cart) {
        PaymentRequest request = createPaymentRequest(cart);
        PaymentResult result = paymentProcessor.process(request);

        return createOrder(cart, result);
    }
}
```

The interface creates a boundary between the checkout workflow and the payment implementation.

However, an interface is not automatically a good abstraction. A useful abstraction should represent a stable concept. Creating an interface for every class can add indirection without reducing coupling.

For example, an interface called `CustomerServiceInterface` with one implementation called `CustomerServiceImpl` may provide little value if it exists only because of a naming convention. The real

question is whether callers need a stable contract that could support different implementations, testing boundaries, infrastructure adapters, or independent evolution.

At a senior level, abstraction is a design decision rather than a mechanical rule.

Inheritance and Substitutability

Inheritance is appropriate when a subtype can genuinely be used anywhere its parent type is expected without changing the correctness of the program.

This is the practical meaning of the Liskov Substitution Principle.

Suppose we define:

```
public interface Account {

    void withdraw(BigDecimal amount);
}
```

Now consider a fixed-deposit account that does not permit withdrawals before maturity:

```
public class FixedDepositAccount implements Account {

    @Override
    public void withdraw(BigDecimal amount) {
        throw new UnsupportedOperationException(
            "Withdrawal is not supported"
        );
    }
}
```

Although the class satisfies the compiler, it does not satisfy the behavioral contract. A caller that accepts an `Account` reasonably expects `withdraw` to be a supported operation.

The subtype has weakened the promise made by the parent type.

A better model separates the capabilities:

```
public interface Account {

    BigDecimal balance();
}

public interface WithdrawableAccount extends Account {

    void withdraw(BigDecimal amount);
}

public final class CheckingAccount
        implements WithdrawableAccount {

    // Withdrawal behavior
}

public final class FixedDepositAccount
        implements Account {

    // No withdrawal promise
}
```

The type system now describes the actual capabilities of each account. Code requiring withdrawal behavior can explicitly depend on `WithdrawableAccount`.

The lesson is that inheritance is not primarily about reusing code. It is about preserving a behavioral contract.

Before creating a subtype, examine whether it:

- Supports every operation promised by the parent.
- Accepts at least the same valid inputs.
- Preserves the parent's invariants.
- Produces results compatible with the caller's expectations.
- Introduces no surprising side effects.

If those conditions are not true, composition or separate interfaces will usually produce a safer design.

Composition as a Way to Combine Behavior

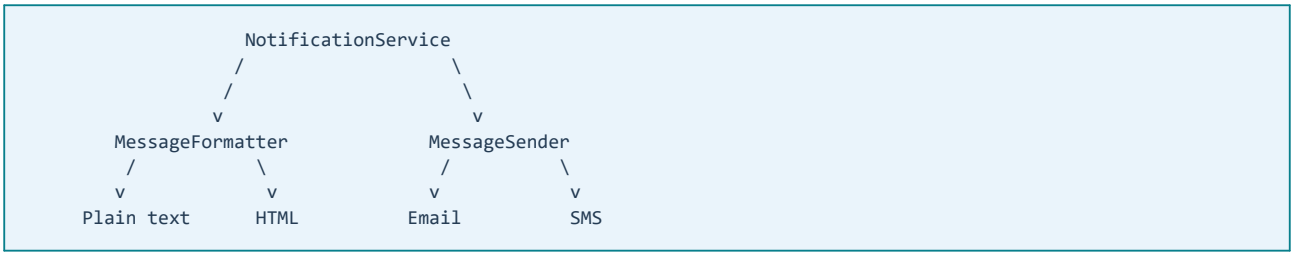
Composition builds an object by connecting smaller collaborators.

Consider a notification service:

```
public interface MessageSender {

    void send(Message message);
}

public interface MessageFormatter {

    String format(Notification notification);
}

public final class NotificationService {

    private final MessageSender sender;
    private final MessageFormatter formatter;

    public NotificationService(
            MessageSender sender,
            MessageFormatter formatter) {

        this.sender = sender;
        this.formatter = formatter;
    }

    public void notify(Notification notification) {
        String body = formatter.format(notification);
        sender.send(new Message(notification.destination(), body));
    }
}
```

The sending mechanism and formatting policy can change independently. Email, SMS, and push delivery do not need to be represented by a deep inheritance hierarchy.

The structure is:



This design allows behavior to be assembled according to the use case.

Composition is often preferred because inheritance creates strong coupling between a parent and its subclasses. A change to a protected method, construction sequence, or parent invariant can affect every subclass.

Inheritance can still be appropriate when the hierarchy represents a stable domain relationship and every subtype preserves the same behavioral contract. The principle is not “never use inheritance.” The principle is to use inheritance for substitutability and composition for flexible behavior assembly.

Polymorphism and Removing Conditional Logic

Polymorphism allows different implementations to satisfy the same contract.

Without polymorphism, business behavior is often centralized in growing conditional statements:

```
public BigDecimal calculateFee(
        String customerType,
        BigDecimal amount) {

    if ("STANDARD".equals(customerType)) {
        return amount.multiply(new BigDecimal("0.03"));
    }

    if ("PREMIUM".equals(customerType)) {
        return amount.multiply(new BigDecimal("0.01"));
    }

    if ("PARTNER".equals(customerType)) {
        return BigDecimal.ZERO;
    }

    throw new IllegalArgumentException(
        "Unsupported customer type"
    );
}
```

This method must be modified whenever a new pricing policy is introduced.

A polymorphic design separates the policies:

```
public interface FeePolicy {

    BigDecimal calculate(BigDecimal amount);
}

public final class StandardFeePolicy
        implements FeePolicy {

    @Override
    public BigDecimal calculate(BigDecimal amount) {
        return amount.multiply(new BigDecimal("0.03"));
    }
}

public final class PremiumFeePolicy
        implements FeePolicy {

    @Override
    public BigDecimal calculate(BigDecimal amount) {
        return amount.multiply(new BigDecimal("0.01"));
    }
}

public final class PartnerFeePolicy
        implements FeePolicy {

    @Override
    public BigDecimal calculate(BigDecimal amount) {
        return BigDecimal.ZERO;
    }
}
```

The application can select the appropriate policy and execute it through the common contract:

```
BigDecimal fee = feePolicy.calculate(amount);
```

This is useful when behaviors change independently or new variants are introduced regularly. It is less useful when the logic is small, stable, and easier to understand as a simple conditional. Replacing every `if` statement with a hierarchy can make a design unnecessarily difficult to follow.

The senior engineering decision is based on the expected direction of change. If new policies are common, polymorphism isolates them. If the set is closed and small, a sealed hierarchy or a switch expression may be clearer.

How These Principles Work Together

The four concepts are not separate rules. They reinforce one another:

```
Encapsulation
protects the object's invariants
        |
        v
Abstraction
exposes a stable contract
        |
        v
Polymorphism
allows multiple valid implementations
        |
        v
Composition or inheritance
organizes how behavior is assembled
```

A well-designed Java component normally has:

- A clear responsibility.
- State that cannot be changed arbitrarily.
- A small public API expressed in domain language.
- Explicit dependencies.
- Predictable error behavior.
- Contracts that implementations can honor.
- Tests based on observable behavior rather than internal methods.

Production Perspective

Poor object design frequently appears in production as operational failure rather than as an obvious object-oriented programming problem.

An object that permits invalid state may eventually produce corrupt database records. A subtype that violates its parent contract may cause failures only for one customer category. A shared mutable object may create intermittent concurrency defects. A weak abstraction may force a single business change across controllers, services, repositories, and message consumers.

These problems are expensive because the visible symptom often appears far from the original design decision.

For that reason, a senior developer evaluates an object by asking:

- What invariant does this object protect?

- Who is allowed to change its state?
- Does its public API describe business operations or expose storage details?
- Can every implementation honor the complete contract?
- Which part of the design is expected to change?
- Is this abstraction reducing coupling or merely adding another layer?
- Is the object safe to share, cache, serialize, or use concurrently?

The answers determine whether the design will remain understandable as the system grows.

Interview-Ready Summary

Object-oriented programming is a method of managing state, behavior, and change through explicit contracts.

Encapsulation protects invariants by controlling how state changes. Abstraction gives callers a stable view without exposing implementation details. Polymorphism allows different implementations to honor the same behavioral contract. Inheritance is appropriate when subtypes are genuinely substitutable, while composition is usually more flexible when independent behaviors must be combined.

The goal is not to maximize the number of interfaces or design patterns. The goal is to create software in which responsibilities are clear, invalid states are difficult to represent, changes remain localized, and behavior remains predictable under production conditions.

Revision Notes

- Encapsulation protects invariants; private fields alone are not sufficient.
- Objects should expose meaningful operations rather than unrestricted setters.
- A useful abstraction represents a stable contract.
- Inheritance requires behavioral substitutability.
- Composition allows independent behaviors to vary separately.
- Polymorphism is valuable when implementations change independently.
- Do not replace simple conditional logic with unnecessary hierarchies.
- Evaluate designs by their expected direction of change.
- Senior-level object design connects code structure to production correctness.